

JudgeMesh 分布式在线判题系统概要设计说明书

案卷号：JudgeMesh-HLD-2026-01

日期：2026-05-06

项目名称：JudgeMesh 分布式在线判题系统

文档类型：概要设计说明书

作者：贾呈煜

完成日期：2026-05-06

签收人：待定

签收日期：待定

修改情况记录

版本号	修改批准人	修改人	修改日期	修改内容	签收人
V1.0	待定	贾呈煜	2026-05-06	根据 baseline《概要设计说明书编写规范》与 docs 现有材料形成正式版	待定

1 引言

1.1 编写目的

本文档用于说明 JudgeMesh 分布式在线判题系统的概要设计方案，明确系统总体架构、运行环境、处理流程、模块结构、接口安排、运行控制、核心数据结构和出错处理策略。本文档承接《软件需求说明书》，为后续详细设计、编码实现、测试计划、部署实施和课程答辩提供设计依据。

1.2 范围

待开发系统名称为 JudgeMesh 分布式在线判题系统。任务提出者为武汉大学《分布式软件原理与技术》课程项目组，开发者为 JudgeMesh 五人项目组，最终用户包括学生、出题人、管理员、运维人员和课程答辩评审。系统运行单位为课程项目演示环境，部署目标为 Kubernetes 集群。

本文档覆盖以下概要设计范围：

- Web 前端、Gateway、业务微服务、判题调度器、判题 Worker、基础设施和可观测性组件。
- 提交到判题的异步流水线。
- 比赛、排行榜、WebSocket 推送和封榜机制。
- Nacos、Sentinel、Seata、RabbitMQ、Redis、MySQL、MinIO、etcd、Kubernetes 等组件的协作。
- 故障场景下的降级、恢复和维护设计。

1.3 定义

术语	定义
Gateway	Spring Cloud Gateway，系统统一入口
user-service	用户、角色、登录、积分服务
problem-service	题目、标签、测试用例元数据服务
submit-service	提交、比赛、排行榜、WebSocket 服务
judge-dispatcher	判题调度服务，负责 MQ 消费、Worker 选择、超时重试和选主
judge-worker	Go 实现的判题执行服务，负责 isolate 沙箱执行
Control Plane	注册配置、限流熔断、事务协调、选主等治理面
Observability	指标、链路追踪、日志和告警组成的可观测体系
DLQ	Dead Letter Queue，死信队列
HPA	Kubernetes 水平自动伸缩

1.4 参考资料

资料名称	来源
《概要设计说明书编写规范》	docs/template/概要设计说明书编写规范.doc
《软件需求说明书编写规范》	docs/template/软件需求说明书编写规范.doc
JudgeMesh 软件需求说明书	docs/design/01-需求文档.md

资料名称	来源
JudgeMesh 服务拆分与接口	docs/design/03-服务拆分.md
JudgeMesh 数据模型与存储	docs/design/04-数据模型.md
JudgeMesh 判题流水线	docs/design/05-判题流水线.md
JudgeMesh 服务治理与容错	docs/design/06-服务治理.md
JudgeMesh 可观测性方案	docs/design/07-可观测性.md
JudgeMesh 部署架构	docs/design/08-部署架构.md
JudgeMesh 测试策略	docs/design/10-测试策略.md

2 总体设计

2.1 需求规定

系统需满足《软件需求说明书》中定义的 P0 和主要 P1 需求。概要设计中重点覆盖以下输入、处理和输出。

2.1.1 主要输入

输入类别	典型输入	来源
用户认证输入	用户名、邮箱、密码、JWT、Refresh Token	浏览器
题目输入	题面、标签、时限、内存、测试用例文件	出题人
提交输入	problemId、contestId、language、code	学生
比赛输入	比赛时间、题目集合、封榜时间、报名用户	管理员或出题人
运维输入	配置、限流规则、混沌实验、发布命令	管理员或运维

2.1.2 主要处理

- 认证与授权：Gateway 验签 JWT，下游服务根据角色控制访问。
- 题目管理：problem-service 管理题目元数据，MinIO 存储用例文件。
- 提交入队：submit-service 写提交记录并投递 RabbitMQ。
- 调度派发：judge-dispatcher 通过 etcd 选主，仅 leader 消费任务并选择 Worker。
- 沙箱判题：judge-worker 使用 isolate 编译运行代码，生成 JudgeResult。

- 结果回写：submit-service 更新 MySQL、Redis 排行榜并通过 WebSocket 推送。
- 观测与恢复：Prometheus、Grafana、SkyWalking、Loki 和 AlertManager 观察系统状态。

2.1.3 主要输出

输出类别	典型输出
用户输出	token、用户资料、权限错误
题目输出	题目列表、题目详情、测试用例 manifest
提交输出	submissionId、判题状态、耗时、内存、错误信息
比赛输出	比赛列表、报名结果、实时排行榜、封榜状态
运维输出	指标大盘、链路追踪、日志、告警、混沌实验结果

2.1.4 主要性能要求

指标	目标
提交接口同步阶段 P99	小于 200 ms
判题端到端 P99	小于 8 s
排行榜查询 P99	小于 50 ms
常态提交吞吐	大于 100 QPS
比赛提交峰值	大于 500 QPS，同步入队不崩溃
dispatcher 选主恢复	10 s 内
Worker 宕机任务重派	30 s 内

2.2 运行环境

2.2.1 软件环境

层次	选型	说明
前端	React、Vite、TypeScript	浏览器端用户界面
Java 运行时	JDK 17	Spring Boot 服务运行环境
Java 框架	Spring Boot 3.5、Spring Cloud 2025、Spring Cloud Alibaba 2025	微服务底座
Worker 运行时	Go 1.22	判题执行节点
数据库	MySQL 8	用户、题目、提交、比赛

层次	选型	说明
缓存	Redis 7	缓存、排行榜、Pub/Sub、防重提交
消息队列	RabbitMQ 3.13+	判题任务削峰和异步派发
对象存储	MinIO	测试用例、头像、备份
共识组件	etcd 3.5	dispatcher 选主和 inflight 记录
注册配置	Nacos 3.x	服务注册和配置中心
限流熔断	Sentinel 1.8.x	网关和服务层保护
分布式事务	Seata 2.x	提交扣积分演示
容器编排	Kubernetes 1.28	部署、服务发现、HPA、滚动升级
可观测性	Prometheus、Grafana、SkyWalking、Loki、AlertManager	指标、链路、日志和告警
混沌工程	ChaosMesh	PodKill、NetworkDelay、CPUSstress

2.2.2 硬件与集群环境 演示环境为 1 master + 3 worker 节点，单节点建议 2C4G 起步，4C8G 更稳妥。

节点	角色	主要负载
node-master	Kubernetes control-plane	kube-apiserver、scheduler、controller、etcd
node-worker-1	通用业务节点 判题节点	Gateway、业务服务、可观测性
node-worker-2		judge-worker，带 role=judge 标签
node-worker-3	混合节点	judge-worker、中间件、可观测性

命名空间规划：

kube-system

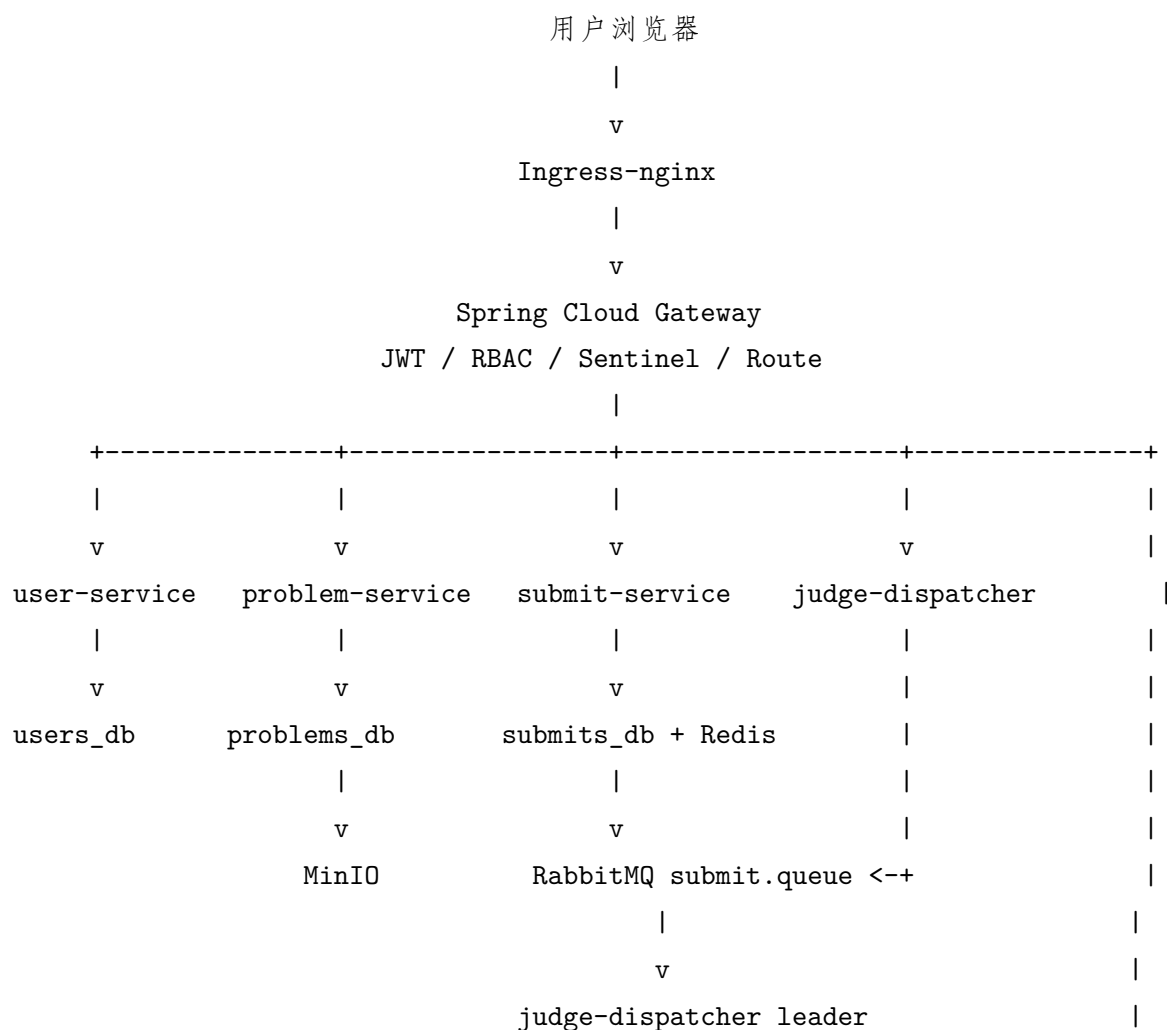
ingress-nginx

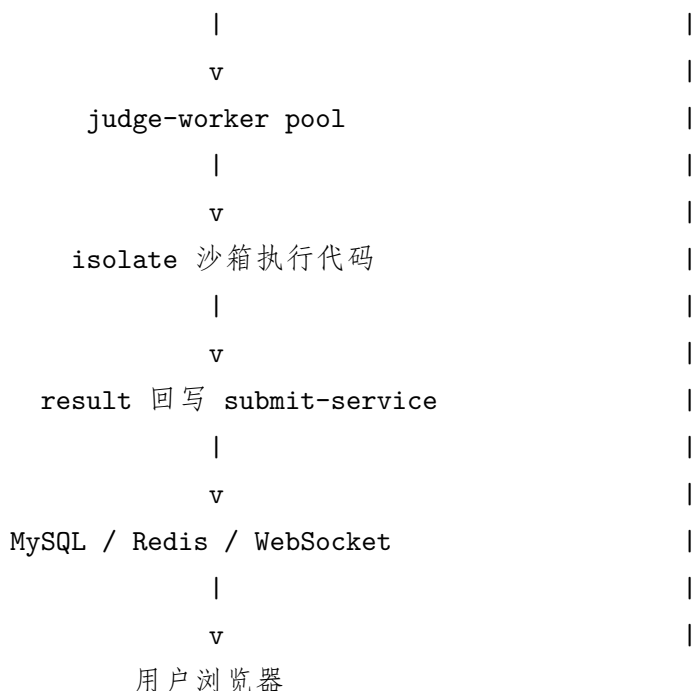
judgemesh

judgemesh-infra

judgemesh-observe

chaos-mesh





治理面：

Nacos: 服务注册 + 配置中心

Sentinel: 限流 / 熔断 / 降级

Seata: 分布式事务协调者

etcd: dispatcher 选主 + inflight 任务记录

ChaosMesh: 混沌实验

观测面:

Prometheus + Grafana: 指标采集与展示

SkyWalking: 全链路追踪

Loki + Promtail: 日志聚合

AlertManager: 告警

2.3.3 提交判题处理流程

1. 学生在前端选择题目、语言并提交代码。
2. Gateway 验证 JWT、执行限流并路由到 submit-service。
3. submit-service 校验题目、比赛、用户状态和防重提交。
4. submit-service 写入 submission, 状态为 PENDING。
5. submit-service 投递 JudgeTask 到 RabbitMQ。
6. judge-dispatcher leader 消费任务。

7. dispatcher 从 Kubernetes 或本地缓存获取 Ready Worker。
8. dispatcher 按负载和健康状态选择 Worker，写 etcd inflight。
9. Worker 下载测试用例，初始化 isolate。
10. Worker 编译和运行用户代码，解析 isolate meta。
11. Worker 生成 JudgeResult 并回写 submit-service。
12. submit-service 更新 submission 状态。
13. submit-service 更新 Redis 排行榜和 Pub/Sub。
14. WebSocket 向浏览器推送判题状态和排行榜变化。

2.3.4 一致性策略 系统不采用单一一致性策略，而是按数据域区分。

数据域	一致性级别	PACELC 取舍	设计说明
用户账号和密码	强一致	PC/EC	安全相关，写 MySQL
用户积分扣减	强一致	PC/EC	Seata AT 演示跨服务事务
题目元数据	强一致 + 缓存	PC/EL	写少读多，写后失效缓存
测试用例文件	强一致	PC/EC	MinIO 私有存储
提交记录	强一致	PC/EC	成绩数据必须可靠
判题结果回写	最终一致	PA/EL	MQ + 重试，允许短暂 Pending
排行榜	最终一致	PA/EL	Redis ZSet，允许秒级滞后
WebSocket 推送	至多一次	PA/EL	推送失败由客户端拉取兜底

2.4 结构

2.4.1 系统元素划分

系统元素	标识符	主要功能	技术
前端	frontend	页面、表单、 WebSocket、交互	React、TypeScript
网关	gateway	路由、JWT、限流、 CORS、统一异常	Spring Cloud Gateway
共享 API	services/api	DTO、Feign、MQ schema、错误码	Java Maven 模块
用户服务	user-service	注册、登录、角色、积 分	Spring Boot、MySQL
题目服务	problem-service	题目 CRUD、用例 manifest、MinIO、缓存	Spring Boot、MySQL、 Redis、MinIO

系统元素	标识符	主要功能	技术
提交服务	submit-service	提交、比赛、排行榜、WebSocket、回写	Spring Boot、MySQL、Redis、RabbitMQ
判题调度器	judge-dispatcher	MQ 消费、选主、Worker 选择、重试	Spring Boot、RabbitMQ、etcd
判题 Worker	judge-worker	编译运行、沙箱、输出比对、指标	Go、isolate
基础设施	infra	K8s、Ingress、中间件、Helm、HPA	Kubernetes、Helm
可观测性	observe	指标、链路、日志、告警	Prometheus、Grafana、SkyWalking、Loki

2.4.2 模块控制关系

```
Browser
-> Gateway
  -> user-service
  -> problem-service
  -> submit-service
    -> user-service
    -> problem-service
    -> RabbitMQ
      -> judge-dispatcher leader
        -> judge-worker
          -> problem-service / MinIO
          -> submit-service
    -> Redis
    -> WebSocket
```

2.4.3 代码目录结构

```
JudgeMesh/
  frontend/
  services/
    api/
    gateway/
```

user-service/
problem-service/
submit-service/
judge-dispatcher/
judge-worker/
infra/
k8s/
helm/
grafana/
chaos/
migrations/
scripts/
tests/
data/
docs/

2.5 功能需求与程序的关系

功能需求	frontend	gateway	user-service	problem-service	submit-service	judge-dispatcher	judge-worker	infra/observe
用户注册登录	✓	✓	✓					
角色权限控制	✓	✓	✓	✓	✓			
题目浏览	✓	✓		✓				
题目管理	✓	✓		✓				
测试用例管理	✓	✓		✓			✓	
代码提交	✓	✓	✓	✓	✓	✓	✓	
多语言判题	✓			✓	✓	✓	✓	
提交记录	✓	✓			✓			

功能需求	frontend	gateway	user-service	problem-service	submit-service	judge-dispatcher	judge-worker	infra/observe
比赛模块	✓	✓	✓	✓	✓			
实时排行榜	✓	✓			✓			
调度器						✓		✓
选主								
可观测性大盘	✓							✓
混沌演示	✓	✓			✓	✓	✓	✓
分布式事务演示	✓	✓	✓		✓			✓

2.6 人工处理过程

人工处理过程	说明
题目审核	出题人创建题目后，管理员可检查题面、样例和测试用例再发布
DLQ 处理	判题任务超过重试次数进入死信队列，需要管理员检查并决定重放或标记 SE
混沌实验触发	答辩演示时由管理员手动触发 PodKill、MQ down 等实验
配置调整	运维人员根据压测结果调整 Sentinel、HPA、Nacos 配置
数据备份恢复	数据异常时运维人员从 MinIO 备份恢复 MySQL 或重建 Redis 排行榜
比赛解榜	封榜结束后管理员手动解榜并发布最终排名

2.7 尚未解决的问题

问题	当前处理	后续方向
isolate 容器需要 privileged 权限	课程演示集群接受该风险	生产环境改用更细粒度 capabilities 或 gVisor
中间件单实例可用性不足	通过备份、告警和文档说明缓解	后续升级 MySQL 主从、Redis Cluster、RabbitMQ 集群
Go Worker 与 Java 链路追踪整合有限	Worker 暴露 traceId 和 Prometheus 指标	后续接入 OpenTelemetry
SPJ 和交互题不支持	本期明确不做	后续通过 Runner 和 Comparator 插件扩展
大规模比赛容量未完全验证	本期压测到课程目标容量	后续用更大集群和更真实负载验证

3 接口设计

3.1 用户接口

3.1.1 页面接口

页面	路径	角色	功能
登录	/login	Guest	登录并保存 token
注册	/register	Guest	创建用户
首页	/	全部	展示入口、热门题目和系统概况
题目列表	/problems	全部	分页、搜索和筛选
题目详情	/problems/:id	Student	查看题面、提交代码
提交记录	/submits	Student	查看个人提交
比赛列表	/contests	Student	查看和报名比赛
比赛详情	/contests/:id	Student	参赛、提交、看榜
个人中心	/profile	Student	查看用户资料
管理后台	/admin	Admin	管理题目、比赛、用户和混沌实验

3.1.2 交互要求

- 登录后由前端在请求头携带 Authorization: Bearer <token>。
- 提交代码后立即展示 PENDING 状态，并订阅 /ws/submission/{id}。

- WebSocket 断线时前端自动重连，并通过 REST 拉取最新状态兜底。
- 表单错误需要显示服务端返回的错误码和友好文案。
- 排行榜首次进入拉取全量，之后接收 delta 更新。

3.2 外部接口

3.2.1 REST API

方法	路径	服务	说明
POST	/api/auth/register	user-service	用户注册
POST	/api/auth/login	user-service	用户登录
POST	/api/auth/refresh	user-service	刷新 token
GET	/api/user/me	user-service	当前用户资料
GET	/api/problem/list	problem-service	题目列表
GET	/api/problem/{id}	problem-service	题目详情
POST	/api/problem	problem-service	创建题目
PUT	/api/problem/{id}	problem-service	更新题目
POST	/api/problem/{id}/testcase	problem-service	上传测试用例
GET	/api/problem/{id}/testcase/manifest	problem-service	Worker 获取用例清单
POST	/api/submit	submit-service	提交代码
GET	/api/submit/{id}	submit-service	查询单个提交
GET	/api/submit/mine	submit-service	查询个人提交
POST	/api/submit/internal/result	submit-service	Worker 回写结果
GET	/api/contest/list	submit-service	比赛列表
POST	/api/contest/{id}/register	submit-service	比赛报名
GET	/api/contest/{id}/rank	submit-service	比赛排行榜
GET	/api/rank/global	submit-service	全站排行榜
GET	/admin/dispatcher/status	pubs-dispatcher	调度器状态
POST	/admin/dispatcher/challenge	pubs-dispatcher	调度器混沌演示

3.2.2 WebSocket 接口

路径	说明
/ws/submission/{id}	推送单个提交状态变化
/ws/contest/{id}/rank	推送比赛排行榜变化

WebSocket 消息格式示例:

```
{
  "type": "SUBMISSION_RESULT",
  "submissionId": 12345,
  "status": "AC",
  "score": 100,
  "timeUsedMs": 128,
  "memoryUsedKb": 24576
}
```

3.2.3 硬件与支持软件接口

接口对象	接口方式	说明
Kubernetes API	HTTPS	dispatcher 获取 Worker Pod 状态, HPA 扩缩容由 K8s 控制
isolate	CLI	Worker 调用 isolate 初始化、运行和清理 sandbox
MinIO	S3 API	上传和下载测试用例
Nacos	HTTP/gRPC 客户端	服务注册、配置读取和热更新
Sentinel Dashboard	HTTP	查看限流和熔断状态
Prometheus	HTTP scrape	抓取应用和基础设施指标

3.3 内部接口

3.3.1 服务间调用矩阵

调用方	被调方	方式	数据
Browser	Gateway	HTTPS / WebSocket	用户请求、状态推送
Gateway	user-service	HTTP LB	认证、用户资料
Gateway	problem-service	HTTP LB	题目
Gateway	submit-service	HTTP LB	提交、比赛、排行榜
submit-service	user-service	Feign	UserDTO、扣积分
submit-service	problem-service	Feign	ProblemDTO、题目限制
submit-service	RabbitMQ	AMQP	JudgeTask
judge-dispatcher	RabbitMQ	AMQP	消费判题任务

调用方	被调方	方式	数据
judge-dispatcher	etcd	gRPC	leader、inflight
judge-dispatcher	judge-worker	HTTP JSON	JudgeTask
judge-worker	problem-service / MinIO	HTTP / S3	测试用例
judge-worker	submit-service	HTTP JSON	JudgeResult
submit-service	Redis	TCP	排行榜、Pub/Sub、防重

3.3.2 JudgeTask

```
{
  "submissionId": 12345,
  "problemId": 100,
  "language": "CPP",
  "code": "int main(){return 0;}",
  "timeLimitMs": 1000,
  "memoryLimitMb": 256,
  "testcaseManifestUrl": "http://problem-service/api/problem/100/testcase/manifest",
  "submittedAt": "2026-05-06T10:00:00+08:00"
}
```

3.3.3 JudgeResult

```
{
  "submissionId": 12345,
  "status": "AC",
  "score": 100,
  "timeUsedMs": 128,
  "memoryUsedKb": 24576,
  "judgeMessage": "",
  "workerId": "judge-worker-2",
  "traceId": "abc123",
  "judgedAt": "2026-05-06T10:00:02+08:00"
}
```

3.3.4 Gateway 路由

```
routes:
  - id: user
    uri: lb://user-service
    predicates: [Path=/api/user/**, /api/auth/**]
  - id: problem
    uri: lb://problem-service
    predicates: [Path=/api/problem/**]
  - id: submit
    uri: lb://submit-service
    predicates: [Path=/api/submit/**, /api/contest/**, /api/rank/**]
  - id: ws
    uri: lb:ws://submit-service
    predicates: [Path=/ws/**]
```

4 运行设计

4.1 运行模块组合

4.1.1 用户登录运行组合

frontend -> gateway -> user-service -> users_db / Redis session

运行说明:

1. 前端提交登录表单。
2. Gateway 路由到 user-service。
3. user-service 校验密码, 签发 token。
4. refresh token 写入 Redis。

4.1.2 题目浏览运行组合

frontend -> gateway -> problem-service -> Redis cache -> problems_db

运行说明:

1. 前端请求题目列表或详情。
2. problem-service 先读 Redis。

3. 缓存未命中时查询 MySQL，回填 Redis。

4.1.3 提交判题运行组合

```
frontend -> gateway -> submit-service -> user-service / problem-service  
-> submits_db -> RabbitMQ -> judge-dispatcher -> etcd -> judge-worker  
-> MinIO -> isolate -> submit-service -> Redis -> WebSocket -> frontend
```

运行说明：

1. 同步阶段以低延迟完成校验、入库和入队。
2. 异步阶段由 dispatcher 和 Worker 完成判题。
3. 结果回写后通过 WebSocket 推送。

4.1.4 比赛排行榜运行组合

```
submit-service -> Redis ZSet -> Redis Pub/Sub -> WebSocket -> frontend
```

运行说明：

1. AC 后 submit-service 更新 Redis ZSet。
2. 若未封榜，发布排行榜 delta。
3. WebSocket 服务向订阅者推送。

4.1.5 混沌演示运行组合

```
admin frontend -> gateway -> admin endpoint / ChaosMesh  
-> Kubernetes PodKill / NetworkDelay  
-> Prometheus / Grafana / AlertManager
```

运行说明：

1. 管理员触发实验。
2. ChaosMesh 注入故障。
3. 系统通过重试、选主、限流或 HPA 恢复。
4. Grafana 展示指标变化。

4.2 运行控制

4.2.1 启动控制 首次部署顺序：

1. Kubernetes 集群就绪。
2. 安装 ingress-nginx 和 cert-manager。
3. 安装 Prometheus、Grafana、SkyWalking、Loki。
4. 安装 ChaosMesh。
5. 部署 MySQL、Redis、MinIO、etcd、RabbitMQ、Nacos、Seata。
6. 导入 Nacos 业务配置。
7. 部署 gateway、user、problem、submit、dispatcher、worker。
8. 部署 Ingress。
9. 执行 smoke test 验证全链路。

4.2.2 发布控制

- 代码合并到主分支后由 CI 运行测试、构建镜像和推送镜像仓库。
- Kustomize 或 Helm 更新镜像 tag。
- Kubernetes Deployment 使用 RollingUpdate。
- readiness probe 通过后新副本接流量。
- 失败时使用 kubectl rollout undo deployment/<name> 回滚。

4.2.3 配置控制

配置类型	存储位置	更新方式
默认配置	应用镜像内 application.yml	随版本发布
业务可调配置	Nacos	热更新
K8s 地址和资源配置	ConfigMap / manifest	kubectl apply
密钥	K8s Secret	手动或密钥管理工具注入
限流熔断规则	Sentinel / Nacos	控制台或配置更新

4.2.4 故障控制

故障	控制方式
Worker 宕机	dispatcher 超时后重新派发，K8s 拉起新 Pod
dispatcher leader 宕机	etcd Lease 过期，follower 竞选新 leader
业务服务实例异常	readiness 失败后流量摘除
MQ 堆积	HPA 扩容 Worker，必要时 Gateway 限流
下游服务慢调用	Sentinel 熔断并执行 fallback

4.3 运行时间

运行过程	目标耗时或资源占用
登录接口	P99 小于 200 ms
题目详情读取	P99 小于 100 ms，缓存命中更低
提交同步阶段	P99 小于 200 ms
判题端到端	P50 小于 2 s，P99 小于 8 s
dispatcher 选主	leader 宕机 10 s 内恢复
Worker 重派	Worker 失联后 30 s 内重新派发
Worker Pod 并发	默认按 CPU limit 的 0.8 折算，本期每 Pod 约 1 个并发 isolate
Worker HPA	minReplicas 3，maxReplicas 20，按队列长度扩缩
Prometheus 抓取	应用指标 15 s，主机指标 30 s
日志保留	Loki 7 天
指标保留	Prometheus 15 天

5 系统数据结构设计

5.1 逻辑结构设计要点

5.1.1 用户域

数据结构	标识符	主要字段
用户	user	id、username、email、password_hash、nickname、balance、total_ac、total_submit
用户角色	user_role	user_id、role

数据结构	标识符	主要字段
Seata 回滚日志	undo_log	branch_id、xid、rollback_info、log_status

5.1.2 题目域

数据结构	标识符	主要字段
题目	problem	id、title、description、time_limit_ms、memory_limit_mb、difficulty、setter_id、published
题目标签	problem_tag	problem_id、tag
用例清单	testcase_manifest	problem_id、case_index、input_object、output_object、score

5.1.3 提交与比赛域

数据结构	标识符	主要字段
提交	submission	id、user_id、problem_id、contest_id、language、code、status、score、time_used_ms、memory_used_kb
比赛	contest	id、title、description、start_time、end_time、freeze_min、created_by
比赛题目	contest_problem	contest_id、problem_id、seq
比赛报名	contest_register	contest_id、user_id、registered_at

5.1.4 Redis 结构

Key	类型	用途
cache:problem:{id}	STRING	题目详情缓存
cache:problem:list:hot	STRING	热门题目列表
cache:user:{id}	STRING	用户公开档案
rank:global	ZSET	全站排行榜
rank:contest:{cid}	ZSET	比赛排行榜
contest:{cid}:freeze	HASH	封榜状态
contest:{cid}:registered	SET	报名用户集合
dedup:submit:{userId}:{problemId}	STRING NX	防 1 秒内重复提交
pubsub:submission:{id}	PUB/SUB	提交状态推送
pubsub:contest:{cid}:rank	PUB/SUB	比赛榜推送

5.1.5 RabbitMQ 结构

结构	名称	说明
Exchange	judgemesh.exchange	direct, durable
Queue	submit.queue	主判题任务队列
Queue	submit.retry.queue	延迟重试队列
Queue	submit.dlq	死信队列
Routing Key	judge.task	新判题任务
Routing Key	judge.retry	重试任务

5.1.6 etcd 结构

```
/judgemesh/  
  dispatcher/  
    leader  
    members/{podName}  
    inflight/{submissionId}  
  config/  
    feature-flags
```

5.2 物理结构设计要点

5.2.1 MySQL 物理设计

表	主键	主要索引	说明
user	id	idx_email, username unique, email unique	用户登录和资料
user_role	user_id, role	无	角色集合
problem	id	idx_setter、 idx_published_diff	题目查询
problem_tag	problem_id, tag	idx_tag	标签筛选
testcase_manifest	id	ux_problem_case	用例清单
submission	id	idx_user_submitted、 idx_problem_status、 idx_contest_user	提交查询
contest	id	idx_time	比赛时间查询
contest_problem	contest_id, problem_id	无	比赛题目
contest_register	contest_id, user_id	无	报名

存储要求：

- 代码字段使用 MEDIUMTEXT，单次提交在应用层限制 64 KB。
- 题面使用 Markdown 存储在 MEDIUMTEXT。
- 提交记录永久保存。
- 所有业务表保留 created_at 和必要的 updated_at。

5.2.2 对象存储物理结构

```
testcases/  
  problem-{id}/  
    1.in  
    1.ans  
    2.in  
    2.ans  
    manifest.json  
  
avatars/
```

user-{id}.png

backups/
daily/
 yyyy-mm-dd/
 users.sql.gz
 problems.sql.gz
 submits.sql.gz

访问控制:

- testcases/ 私有, Worker 使用预签名 URL。
- avatars/ 可公开读。
- backups/ 仅管理员和备份任务访问。

5.2.3 Redis 排行榜计分 全站榜:

score = totalAC * 1_000_000 - totalAcceptedTimeSeconds

比赛榜:

solved = 已 AC 题数
penalty_min = sum(每道 AC 题首次 AC 时间分钟 + 错误提交数 * 20)
score = solved * 100_000 - penalty_min

5.3 数据结构与程序的关系

数据结构	user-service	problem-service	submit-service	judge-dispatcher	judge-worker	infra/observe
user	✓		✓ 读			
user_role	✓					
undo_log	✓		✓			
problem		✓	✓ 读			
problem_tag		✓				
testcase_manifest		✓			✓ 读	
submission			✓		✓ 回写触发	
contest			✓			

数据结构	user-service	problem-service	submit-service	judge-dispatcher	judge-worker	infra/observe
contest_problem			✓			
contest_register			✓			
Redis cache	✓	✓	✓			
Redis ZSet			✓			
RabbitMQ queues			✓ 写	✓ 读写		✓ 观测
MinIO objects		✓ 写			✓ 读	✓ 备份
etcd				✓		✓ 观测
leader/inflight						

6 系统出错处理设计

6.1 出错信息

错误码	HTTP 状态	含义	处理方法
AUTH_REQUIRED	401	未登录	前端跳转登录
TOKEN_EXPIRED	401	Token 过期	前端尝试 refresh
FORBIDDEN	403	权限不足	提示用户无权限
VALIDATION_FAILED	400	参数校验失败	显示字段错误
PROBLEM_NOT_FOUND	404	题目不存在	返回题目列表
CONTEST_NOT_ACTIVE	409	比赛不可提交	提示比赛时间状态
DUPLICATE_SUBMIT	429	1 秒内重复提交	提示稍后再试
RATE_LIMITED	429	触发限流	前端提示系统繁忙
SUBMISSION_NOT_FOUND	404	提交不存在或无权限	提示无记录
JUDGE_TIMEOUT	504	判题超时	dispatcher 重试或标记 SE
WORKER_UNAVAILABLE	503	无可用 Worker	任务重试，入口限流
MQ_UNAVAILABLE	503	MQ 不可用	降级或拒绝提交
SYSTEM_ERROR	500	未预期异常	记录 traceId 并告警

判题状态信息：

状态	含义	前端展示
PENDING	等待派发	等待中
JUDGING	正在判题	判题中
AC	答案正确	Accepted
WA	答案错误	Wrong Answer
TLE	超出时间限制	Time Limit Exceeded
MLE	超出内存限制	Memory Limit Exceeded
RE	运行时错误	Runtime Error
CE	编译错误	Compile Error
SE	系统错误	System Error

6.2 补救措施

6.2.1 后备技术

数据或服务	后备措施
MySQL	每日 mysqldump 到 MinIO，必要时恢复
Redis 排行榜	从 MySQL submission 重建
RabbitMQ	durable queue 和 persistent message，DLQ 保存最终失败任务
MinIO 测试用例	MinIO 版本控制和定期备份
Nacos 配置	Nacos 版本回滚，manifest 中保留默认配置
应用镜像	镜像仓库保留 git-sha tag，可回滚到旧版本

6.2.2 降效技术

故障场景	降效策略
Worker 池不足	提交继续入队，网关降低提交速率
Redis 不可用	排行榜暂不可用，提交结果仍写 MySQL
WebSocket 不可用	前端改用轮询查询提交状态
user-service 慢调用	Sentinel fallback 返回占位公开资料
problem-service 缓存失效	直接查 MySQL，必要时限流
SkyWalking 或 Loki 不可用	不影响业务，保留本地标准输出日志

6.2.3 恢复及再启动技术

故障	恢复方式
业务服务崩溃	Kubernetes 自动重启, readiness 通过后接流量
Worker 崩溃	dispatcher 超时重派, Deployment 拉起新 Pod
dispatcher leader 崩溃	etcd Lease 过期后 follower 抢锁
发布失败	kubect1 rollout undo 回滚
配置错误	Nacos 配置版本回滚
数据库迁移失败	Flyway repair 或回滚脚本, 停止相关服务写入
队列堆积	HPA 扩容 Worker, 必要时限制提交入口

6.3 系统维护设计

6.3.1 可观测性维护点

维护对象	检测点
Java 服务	/actuator/health、/actuator/prometheus
Go Worker	/healthz、/metrics
dispatcher	leader 状态、lease 剩余时间、inflight 数
RabbitMQ	queue ready、unacked、publish rate、consume rate
Redis	命中率、内存、慢查询、ZSet 大小
MySQL	连接数、慢查询、磁盘、备份状态
Kubernetes	Pod Ready、重启次数、CPU、内存、HPA 副本数

6.3.2 日志与链路维护

- 所有请求生成或透传 traceId。
- 关键业务事件记录 INFO 日志。
- 可恢复异常记录 WARN 日志。
- 系统异常记录 ERROR 日志并触发告警。
- 提交记录保存 traceId, 便于从提交详情跳转到链路追踪。

6.3.3 配置和发布维护

- Nacos 保存服务配置版本, 重要配置变更需记录变更人和原因。

- Secret 不进入 Git，部署时注入。
- CI 必须执行 Java、Go 和前端测试。
- 数据库迁移由 Flyway 管理，迁移脚本随代码评审。
- Grafana Dashboard JSON 纳入版本控制。

6.3.4 例行维护任务

周期	任务
每次发布前	运行单元测试、集成测试、smoke test
每日	备份 MySQL，检查核心告警
每周	检查 Redis 内存、RabbitMQ 堆积、日志容量
答辩前	跑压测、混沌演练、确认 Grafana 和 SkyWalking 数据

6.3.5 已知风险维护说明

风险	维护策略
privileged Worker 容器	限制集群用途，隔离 judge 节点，后续改用更细粒度权限
中间件单实例	每日备份和快速重建脚本，答辩时说明限制
云资源有限	演示时控制 HPA 上限和压测强度
MQ 或 Redis 故障影响体验	提供降级提示和恢复脚本
配置漂移	Nacos 配置导出入库，关键配置纳入 review

6.4 演进路线与高可用增强

本期项目以 4 周课程交付为目标，优先实现完整主链路和可演示的分布式能力。以下能力在概要设计中保留扩展点，但不作为本期 P0 验收范围。

阶段	演进项	触发条件	设计预留
Phase 2	MySQL 主从和读写分离	单库读 QPS 大于 5000 或备份恢复压力明显	Repository 层封装数据访问，数据库连接由配置中心控制

阶段	演进项	触发条件	设计预留
Phase 2	Redis Cluster	单实例内存大于 8 GB 或排行榜访问成为瓶颈	Redis key 按业务域命名，避免跨 key 强事务依赖
Phase 2	RabbitMQ 集群	提交任务队列成为单点风险	消息 schema 独立在共享 API 模块，业务侧不依赖具体队列部署形态
Phase 2	Seata Server 集群	分布式事务演示路径需要更高可用	Seata 客户端通过 Nacos 发现 TC 地址
Phase 3	RabbitMQ 迁移 RocketMQ 或 Kafka	提交峰值大于 10K QPS	submit-service 通过消息发布接口隔离 MQ 实现
Phase 3	Service Mesh	微服务数量大于 15 或治理规则复杂化	服务间 HTTP 接口保持标准化，便于接入 Istio
Phase 3	多沙箱 Runner	需要 SPJ、交互题或更强隔离	judge-worker 内部保留 Runner 和 Comparator 抽象